

```
# main.py
# Run locally with:  uvicorn main:app --
reload --port 8000
# Then in the frontend, set USE_FASTAPI =
true and API base URL to
# http://localhost:8000 (see api-
client.ts).

from datetime import date, timedelta
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import
CORSMiddleware

from models import (
    SearchQuery,
    SearchResult,
    TrainRecommendation,
    AlternateStation,
    AlternateDate,
    SeatStatus,
    ALL_CLASSES,
)
from prediction import
heuristic_confirmation_score,
recommendation_score
from db import fetch_trains_for_route

app = FastAPI(title="Smart Ticket AI –
Phase 1 API")
```

```

# Allow the Lovable-hosted frontend (and
local dev) to call this API.
# Tighten this to your real frontend domain
before shipping to production.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

def _days_before(journey_date_str: str) ->
int:
    try:
        d =
date.fromisoformat(journey_date_str)
    except ValueError:
        return 7
    return max(0, (d - date.today()).days)

def _seat_status(confirm_probability: int)
-> SeatStatus:
    if confirm_probability >= 80:
        return SeatStatus(label=f"AVL
{confirm_probability}", tone="success")
    if confirm_probability >= 55:
        return SeatStatus(label=f"WL {100 -
confirm_probability}", tone="warning")
    return SeatStatus(label=f"WL {100 -
confirm_probability}", tone="danger")

```

```

def _build_availability(journey_date: str,
                        days_before: int) -> dict:
    return {
        cls:
        _seat_status(heuristic_confirmation_score(c
ls, journey_date, days_before))
        for cls in ALL_CLASSES
    }

@app.post("/search",
response_model=SearchResult)
def search(query: SearchQuery) ->
SearchResult:
    train_rows =
fetch_trains_for_route(query.source,
query.destination)
    if not train_rows:
        raise HTTPException(
            status_code=404,
            detail="No trains found for
this route yet. Try nearby major
stations.",
        )

    days_before = _days_before(query.date)

    recommendations = []
    for t in train_rows:
        confirm =
heuristic_confirmation_score(
            query.travelClass, query.date,
days_before
        )

```

```

        # Rough duration-in-minutes parse
        for the ranking penalty; falls back
        # to a neutral value if the stored
        format can't be parsed.
        try:
            h, m =
t["duration"].lower().replace("h",
 "").replace("m", "").split()
            duration_minutes = int(h) * 60
+ int(m)
        except Exception:
            duration_minutes = 300

        rec = TrainRecommendation(
            trainName=t["train_name"],
            trainNumber=t["train_number"],
            departure=t["departure_time"],
            arrival=t["arrival_time"],
            duration=t["duration"],
            confirmProbability=confirm,

recommendationScore=recommendation_score(co
nfirm, duration_minutes, fare=800),
            bestClass=query.travelClass,
            reason=(
                f"{confirm}% estimated
confirmation chance in {query.travelClass}
"

                f"based on booking
{days_before} day(s) before journey."
            ),

availability=_build_availability(query.date
, days_before),

```

```

    )
    recommendations.append(rec)

    recommendations.sort(
        key=lambda r: r.recommendationScore
        * 0.6 + r.confirmProbability * 0.4,
        reverse=True,
    )
    best, *rest = recommendations

    alternate_stations = [
        AlternateStation(
            code=f"
{query.source[:3].upper()}{i+1}",
            name=f"{query.source} {name}",
            distanceKm=10 + i * 12,
            extraTravel=f"{20 + i * 10}
min",
            availability=_seat_status(

heuristic_confirmation_score(query.travelCl
ass, query.date, days_before + 1)
        ),
    )
    for i, name in enumerate(["Central
Jn", "Cantt", "City Jn", "Terminus"])
    ]

    today = date.fromisoformat(query.date)
    if query.date else date.today()
    alternate_dates = []
    for i in range(7):
        d = today + timedelta(days=i)
        db_ = max(0, (d -

```

```

date.today()).days)
        confirm =
heuristic_confirmation_score(query.travelClass, d.isoformat(), db_)
        alternate_dates.append(
            AlternateDate(
                date=d.isoformat(),
                weekday=d.strftime("%a"),

status=_seat_status(confirm),
                fare=950 + i * 120,
            )
        )

        ai_insights = [
            f"Historical patterns suggest
{best.trainName} confirms ~
{best.confirmProbability}% "
            f"of {query.travelClass} bookings
this far before departure.",
            "Booking earlier generally raises
confirmation probability – see the date "
            "comparison below.",
            f"{alternate_stations[0].name}
shows comparable availability with only "
            f"
{alternate_stations[0].extraTravel} extra
travel time.",
        ]

        return SearchResult(
            query=query,
            best=best,
            otherTrains=rest,

```

```
alternateStations=alternate_stations,  
    alternateDates=alternate_dates,  
    aiInsights=ai_insights,  
    )
```

```
@app.get("/health")  
def health():  
    return {"status": "ok"}
```